arXiv:2408.12570v1 [cs.CL] 22 Aug 2024

Jamba-1.5: Hybrid Transformer-Mamba Models at Scale

Jamba Team

AI21 labs

Abstract

We present Jamba-1.5, new instruction-tuned large language models based on our Jamba architecture. Jamba is a hybrid Transformer-Mamba mixture of experts architecture, providing high throughput and low memory usage across context lengths, while retaining the same or better quality as Transformer models. We release two model sizes: Jamba-1.5-Large, with 94B active parameters, and Jamba-1.5-Mini, with 12B active parameters. Both models are fine-tuned for a variety of conversational and instruction-following capabilties, and have an effective context length of 256K tokens, the largest amongst open-weight models. To support cost-effective inference, we introduce ExpertsInt8, a novel quantization technique that allows fitting Jamba-1.5-Large on a machine with 8 80GB GPUs when processing 256K-token contexts without loss of quality. When evaluated on a battery of academic and chatbot benchmarks, Jamba models achieve excellent results while providing high throughput and outperforming other open-weight models on long-context benchmarks. The model weights for both sizes are publicly available under the Jamba Open Model License and we release ExpertsInt8 as open source.

Models: https://huggingface.co/ai21labs

1 Introduction

This paper introduces Jamba-1.5, two new large language models based on our Jamba architecture [24], which are available for public use. Jamba-1.5-Mini is an updated and instruction-tuned version of our earlier Jamba release [24]. Like its smaller sibling, Jamba-1.5-Large is a hybrid architecture that mixes Transformer [36] and Mamba [13] layers, with a mixture-of-experts (MoE) module [8, 34]. Since the introduction of Jamba, similar efforts have confirmed the benefits of combining Transformer and state-space-models at a scale of up to 8B parameters [6, 37]. Jamba-1.5-Large demonstrates the benefits of this architecture at a much larger scale. It has 94B active parameters, out of a total of 398B parameters. Even at this large size, the model can fit on a single machine with 8 80GB GPUs when processing a context of 256K tokens, thanks to the efficiency of the Jamba architecture in addition to a novel quantization technique we have developed, as described in Section 3.1.

Both Jamba-1.5-Mini and Jamba-1.5-Large are instruction-tuned models, having undergone post-training to provide them with various capabilities. Our evaluations across a wide range of benchmarks show that they perform comparably to models at their size, while offering the efficiency benefits of the Jamba architecture. In particular, Jamba-1.5 models shine at long-context evaluations, making them the only models with an effective length of 256K on the RULER benchmark, while offering 10x reduction in KV cache memory as well as superior throughput and latency.

We make the Jamba-1.5 models available under the Jamba Open Model License:
https://www.ai21.com/licenses/jamba-open-model-license.
The models are publicly available:
Jamba-1.5-Mini: https://huggingface.co/ai21labs/AI21-Jamba-1.5-Mini
Jamba-1.5-Large: https://huggingface.co/ai21labs/AI21-Jamba-1.5-Large

2 Model Architecture

Jamba-1.5-Large is based on Jamba [24], our hybrid decoder architecture that mixes Transformer layers [36] with Mamba layers [13], a state-space model (SSM) [14, 15], in addition to a mixture-of-experts (MoE) module [8, 34]. See [24] for a detailed description of this architecture.

During our work on Jamba [24], we found that the combination of Transformer, Mamba, and MoE elements facilitates balancing desiderata of throughput, memory usage, and quality. Jamba-1.5-Large demonstrates this flexibility at a larger scale.

Jamba-1.5-Large follows the same Jamba structure but with a larger capacity. It has 94B active parameters and 398B total parameters. It has 9 blocks, with each block having the following specs:

- $l = 8$ layers in each block.
- $a : m = 1 : 7$ ratio of attention-to-Mamba layers. This ratio was found optimal in our work on Jamba [24] and similar ratios was also confirmed as successful in follow-up work [6, 37].
- MoE is used instead of a single MLP every $e = 2$ layers. There are $n = 16$ experts, and we select the top $K = 2$ at each token.
- The hidden state dimensionality is 8192.
- The number of attention query heads is 64 and the number of KV heads is 8.

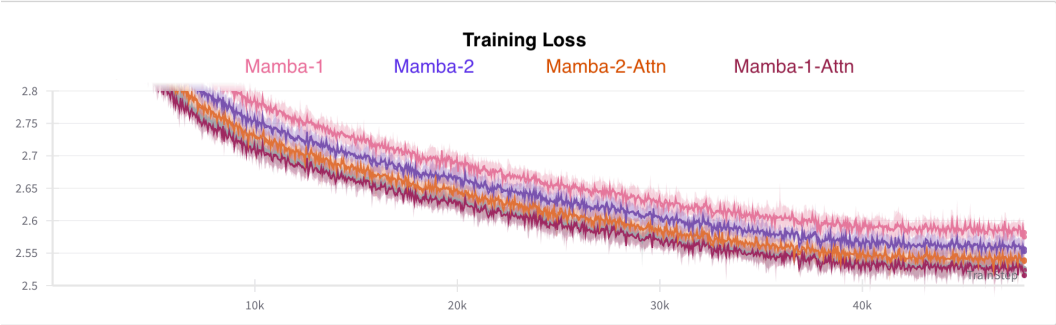
Table 1 compares the Jamba-1.5 models to publicly available models of similar sizes. Jamba-1.5-Mini has a similar number of active parameters as Mixtral 8x7B, while Jamba-1.5-Large's active parameter count is between LLaMA-3.1-70B and Mistral-Large-2. At the same time, both our Jamba models have a much smaller KV cache memory usage (at 256K tokens) compared to all other models, with roughly an order of magnitude reduction compared to their respective counterparts.

With these settings, and our specialized quantization (Section 3.1), Jamba-1.5-Large can be served on a single machine with 8 80GB GPUs with context lengths up to 256K tokens.

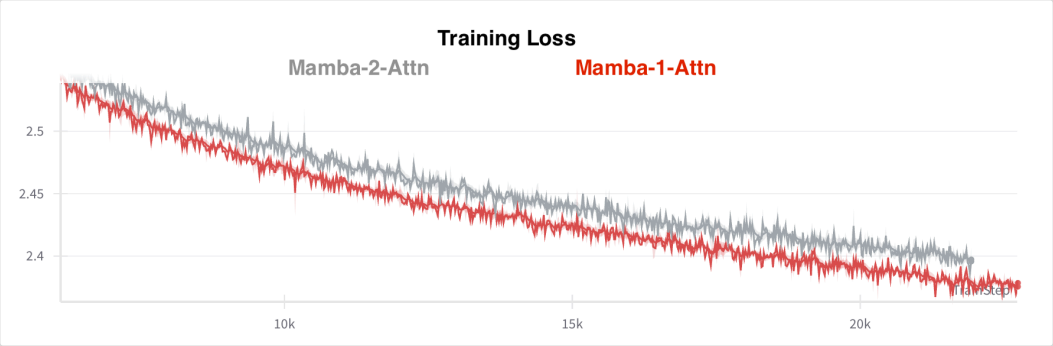
	Available params	Active params	KV cache (256K context, 16bit)
Mistral	7.2B	7.2B	32GB
Mixtral 8x7B	46.7B	12.9B	32GB
LLaMA-3.1 8B	8B	8B	32GB
Mixtral 8x22B	141B	39B	56GB
Mistral-Large-2	123B	123B	88GB
LLaMA-3.1 70B	70B	70B	80GB
LLaMA-3.1 405B	405B	405B	252GB
Jamba-1.5-Mini	52B	12B	4GB
Jamba-1.5-Large	398B	94B	9GB

Table 1: Comparison of Jamba-1.5-Mini, Jamba-1.5-Large and recent open models in terms of total available parameters, active parameters, and KV cache memory on long contexts. Jamba-1.5-Mini and Jamba-1.5-Large provide substantial reductions in the KV cache memory requirements.

For this release, we experimented also with Mamba-2 [6], a faster and improved version of Mamba, which was reported to outperform Mamba and Transformers separately. However, as Figure 1 shows, we found that in a hybrid architecture, the Mamba-1-Attention combination works better than Mamba-2-Attention, so we use Mamba-1 in Jamba-1.5-Large. (We also found the hybrid architecture to outperform pure Mamba-2.) We hypothesize this is because some of the advantages of Mamba-2 over Mamba-1, in particular the ability to use a much larger state size, are less significant when we have full attention layers interleaved between the Mamba layers, as they can pool information from the entire context.



(a) 350M.



(b) 1.3B.

Figure 1: Comparison of Mamba-1, Mamba-2, Mamba-1-Attention, and Mamba-2-Attention on models trained for 100B tokens. While Mamba-2 outperforms Mamba-1 without attention, the hybrid Mamba-1-Attention performs better.

3 Serving Considerations and Improvements

We share a few insights and improvements we have introduced to allow for efficient serving of Jamba models at a large scale.

3.1 ExpertsInt8 Quantization

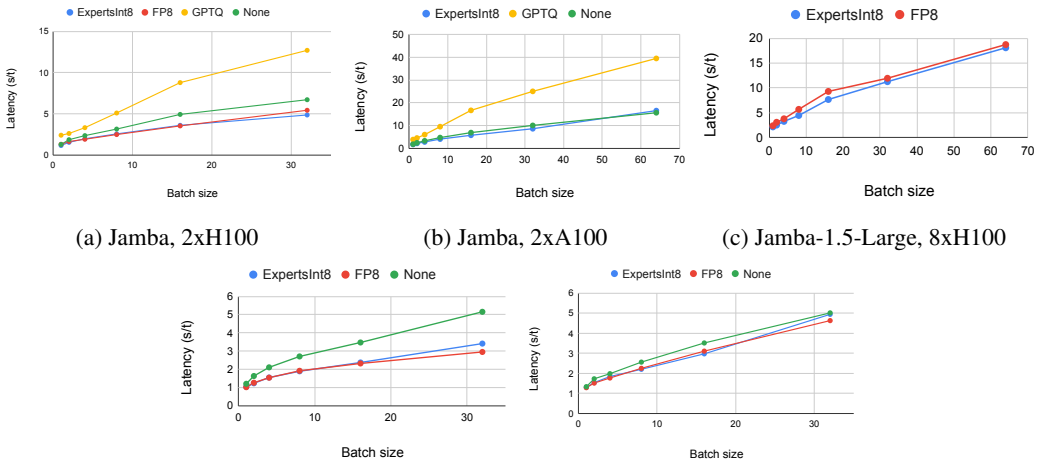
To support efficient serving of Jamba-1.5-Large, we developed a new quantization technique, which we dub ExpertsInt8. We observe that over 85% of the model weights are in the MoE layers, and over 90% are in MoE or MLP layers. We wish to quantize these weights while still enjoying the benefits of fast BF16 kernels. To do so, we quantize the MoE and MLP weights to INT8, save them in INT8, and dequnatize them back to BF16 before the actual computation. Importantly, the dequantization step happens directly inside the `fused_moe` kernel in vLLM [18]. In this way, the dequantization process adds negligible overhead, and even leads to improved latency over BF16.[1] We have contributed our modified `fused_moe` kernel to vLLM.[2]

[1]We attribute this to the the kernel operating on relatively small blocks of weights and activations, which it moves from GPU HBM to SRAM prior to performing the computations. In our implementation, the weights move from HBM to SRAM when they are in int8, so it takes less time as their memory footprint is cut by half.

[2]Pull request here: https://github.com/vllm-project/vllm/pull/7415

Our ExpertsInt8 method has several advantages. First, it is fast; quantization only takes a few seconds at model loading. Second, unlike most other techniques in vLLM, it does not rely on calibration, which can take hours or days and can be unstable. Third, we can still use BF16 to hold large activations. Fourth, it is available to use on A100 GPUs, unlike FP8, which is only available on H100. Finally, our quantization matches FP8 in latency, while surpassing other quantization techniques, without a loss in quality.

Figure 2 compares the latency with different quantization techniques using Jamba-1.5-Mini, Jamba-1.5-Large, and two Mixtral models (8x78B and 8x22B). On H100 GPUs, ExpertsInt8 matches the latency of FP8. On A100, where FP8 is unavailable, ExpertsInt8 is an attractive technique, outperforming GPTQ [9] by a large margin. Together with the advtanages of ExpertsInt8 explained above, this makes it an attractive quantization technique for serving large MoE models.



(a) Jamba, 2xH100 (b) Jamba, 2xA100 (c) Jamba-1.5-Large, 8xH100

(d) Mixtral-8x7B, 2xH100 (e) Mixtral-8x22B, 8xH100

Figure 2: Comparison of different quantization techniques, showing end-to-end latency with 1024-token context and 128-token decoding. ExpertsInt8 performs similar to FP8, while being fast and simple to apply and still allowing BF16 activations, as well as applicable to A100 GPUs, where FP8 is unavailable.

3.2 Activation Loss

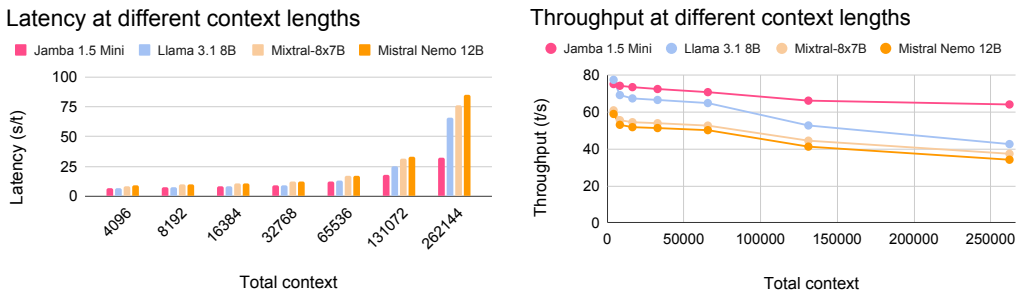
During pre-training, we found that certain activations, namely outputs of specific experts as well as the the output of the last Mamba layers, were gradually increasing in magnitude for certain input tokens, eventually reaching values as high as 4×10^6. Although we did not find this to hurt the pre-training itself, which was done in BF16 precision, the magnitude of the activations could cause numerical issues during inference as some quantization libraries support only FP16 precision for activations, which has a maximum range of 64K.

To alleviate these concerns, we added an "Activation Loss" term, proportional to the mean-square of activations in the forward pass, with a configurable α factor, which penalizes larger activation values. We found via experimentation that this auxilary loss has no affect on the training even with α values up to at least 10^{-3}. For Jamba-1.5-Large, we used $\alpha = 10^{-5}$ which was enough to reduce the activations to an acceptable range (2K-3K max). Moreover, adding this auxilary loss reduced the activations almost instantly, allowing it to be added only towards the end of the training without any affect on training speed and quality.

To validate this approach, we ran our full evaluation suite on the model using FP16 activations and obtained the same results as the BF16 evaluations without any nans/overflows.

4 Throughput and Latency Analysis

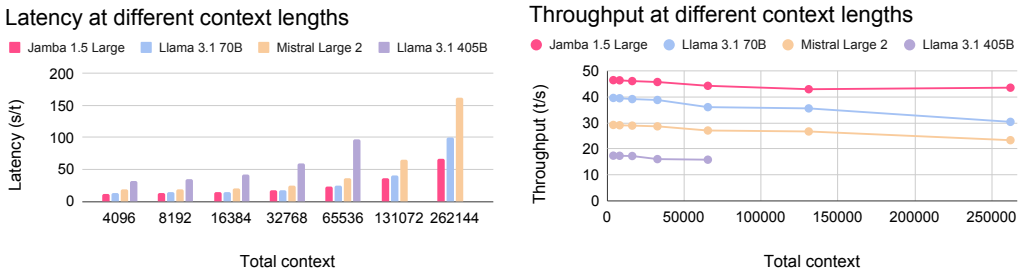
Thanks to the hybrid Jamba architecture, our Jamba-1.5 models provide excellent throughput and latency. Figures 3 and 4 show this for Jamba-1.5-Mini and Jamba-1.5-Large, respectively. As shown in the figures, our models obtain much better latency and throughput than similarly-sized models. Their advantage shines at long contexts, with substantial gaps. Importantly, Jamba-1.5-Large runs efficiently even at long contexts, where the large LLaMA3-405B cannot run on the same hardware.



(a) Jamba-1.5-Mini, end-to-end latency.

(b) Jamba-1.5-Mini, output tokens throughput.

Figure 3: Comparison of Jamba-1.5-Mini with other models in terms of latency and throughout. All measurements were done on 2xA100 80GB GPUs, with batch size 1, and output length 512 tokens. Jamba-1.5-Mini exhibits better latency, especially at large contexts, with only a slight reduction in output tokens throughput.



(a) Jamba-1.5-Large, end-to-end latency.

(b) Jamba-1.5-Large, output tokens throughput.

Figure 4: Comparison of Jamba-1.5-Large with other models in terms of latency and throughout. All measurements were done on 8xA100 80GB GPUs, with batch size 1, and output length 512 tokens. Jamba-1.5-Large exhibits better latency, especially at large contexts, with only a slight reduction in output tokens throughput. The LLaMA-3.1-405B results truncate at 64K because the model is too large to fit context lengths greater than $\approx 100K$ tokens on 8 80GB GPUs.

5 Training

5.1 Training Infrastructure and Data

Jamba-1.5-Large was trained on NVIDIA H100 GPUs using our in-house proprietary framework, which includes FSDP, tensor parallelism, sequence parallelism, and expert parallelism. For the latter we have adapted MegaBlocks [10].

5.2 Training Stages

The model was trained in three stages. During pre-training, it was first trained on an in-house dataset last updated in March 2024. Our pre-training dataset is a mixture of publicly available web documents, code, books and scientific articles. Our pre-processing pipeline includes parsing, quality filters, and deduplication. To make the best use of publicly available data, we developed our own in-house parser, and used it to extract text and formatting. The exact data mixture was determined through various

ablations. This stage included multilingual data with emphasis on the following languages: English, Spanish, French, Portueguse, Italian, Dutch, German, Arabic, and Hebrew. It was then trained for a short phase of mid-training with a high proportion of long documents to emphasize its long-range capabilities. Finally, the model went through post-training, described in the next section.

5.3 Post-training

Our approach to post-training aims to achieve two objectives simultaneously: (*i*) provide the model with various skills and conversational capabilities; (*ii*) retain capabilities from pre-training and especially the long-context capabilities from mid-training. These two objectives are partly conflicting, since most of the available post-training datasets consist of relatively short examples.

Given these considerations, our post-training process involves supervised fine-tuning [32, 39] on high-quality conversational data, skill-specific data, and long-context data. Mixing these different types of data aims to retain long-context capabilities and acquire desired skills. As shown in the evaluations below, we find that our models perform very well in long-context evaluations.

When performing supervised fine-tuning, we make heavy use of synthetic data, as is common in recent foundation models [7] and reflecting our approach for constructing structured data for building compound AI systems [20]. We developed multiple different data synthesis pipelines, targeting different model capabilities. All pipelines apply the following pattern: (i) Sample or generate prompts in a target distribution; (ii) Generate responses from language models; (iii) Filter or rank responses by quality according to automatic validation and scoring; and (iv) Post-edit to remove artifacts and fit desired formatting. We use different models, prompting, sampling, filtering and editing for different data pipelines that compose the final data mixes.

We picked our final training recipes (data mix and hyperparameters) based on a battery of mostly internal automatic metrics. Both Jamba-1.5 models are fine-tuned with the same control tokens and formatting template, which we provide as a part of our release as a HF-compatible tokenizer and chat template; see the model card for details.

We give several notable examples of synthetic data generation:

Table-based QA. We generate tabular data and accompanying question-answer pairs, as demonstrated in our work on table understanding [20]. We then convert the tables into natural language paragraphs using a language model. Our generated training examples include extraction, aggregation, and attribution tasks vis-a-vis text corresponding to specific rows or columns in a given table.

Document QA. Given a document, we prompt a language model to generate question-answer pairs, for both single and multiple paragraphs. We sometimes embed these examples within longer context by adding similar texts, to encourage long-context understanding with attribution.

Tool use. We use the open-source Glaive function-calling dataset [1] as a starting point, filtered with various heuristics and validations on the output schemas. To support parallel function calling, we first generate multiple valid parameter assignments for each function in Glaive. Next, we sample subsets of these valid parameter assignments, for the same function and across different functions, to generate user requests corresponding to the set of function calls. Finally, we prompt a function-calling language model to respond to these generated user requests and retaineonly responses where the function calls matched the original parameter assignments.

Steerability. We defined a set of instructions that can be easily validated and synthesized prompts that include a generic document-drafting task with one or more constraints added to it. We generated completions for these prompts from a language model and used rejection sampling based on the validations of our fine-grained instructions plus a general-purpose reward model. To support instructions in system messages, we chose multiple prompts of this kind that share a fine-grained instruction instance and reformatted these prompts into a multi-turn conversation, with the instruction moved to the system message.

5.4 Some Observations

We share a few observations from the development of Jamba-1.5. While these are not fully explored, we hope they would inspire the community to look further into these issues.

First, while we included only a very small fraction of non-english data, for a few languages and only for specific skills in the post-training phase, our Jamba-1.5 models perform quite well in multiple languages. We did include multilingual data in the pre-training phase, as mentioned above. Thus we speculate that the models are able to use the learned knowledge from that phase when being post-trained mostly in English.

Second, our efficient Jamba architecture lowers the cost of fine-tuning on long contexts, allowing us to experiment more with a given budget. Thus we could experiment with multiple different training recipes at the post-training stage.

Finally, while preference tuning algorithms like PPO [33] or DPO [29] improve alignment between model outputs and human intent, we found that the combination of careful synthetic data generation, data filtering, and supervised fine-tuning is crucial for obtaining a strong post-trained model.

6 Evaluation

While we believe benchmarks are only partly correlated with success of real applications and user satisfaction, we report results on key public benchmarks. First, we report results on standard academic benchmarks. Then, we evaluate the model on chatbot benchmarks. Finally, we evaluate Jamba-1.5-Large on several long-context evaluations and a multilingual evaluation.

We compare with recent open-weight models of the same size range: LLaMA-3.1 70B and Mistral-Large-2-123B when comparing with Jamba-1.5-Large; LLaMA-3.1-8B and Gemma-2-9B when comparing with Jamba-1.5-Mini.

6.1 Academic Benchmarks

We report results with a wide range of standard academic benchmarks: MMLU [16], MMLU-Pro [38], GPQA [31], ARC-Challence [5], BBH [35], and HumanEval [4]. We also evaluate on the IFEval instruction following dataset [42] and the BFCL v1 function calling dataset [40]. Finally, we report safety evaluations on RealToxicity [12] and TruthfulQA [26].

Table 2 compares Jamba-1.5-Large to several publicly available models at similar sizes. All results are either taken from official sources or evaluated by us, as indicated in the table.[3] We observe that the Jamba-1.5 models perform similarly to recent state-of-the-art publicly available models on standard academic benchmarks, including knowledge, reasoning, instruction following and function calling capabilities. We also observe similar safety metrics as those reported in the literature. We refer to Section 7 for more information about our general approach for safety and alignment of models.

Importantly, the Jamba-1.5 models achieve these results while providing much better throughput and latency, as discussed above.

Benchmark	Metric	**Jamba-1.5 Mini**	LLaMA-3.1 8B	Gemma-2 9B	**Jamba-1.5 Large**	LLaMA-3.1 70B	Mistral-L-2 123B
MMLU	5-shot	69.7	69.4	71.3	80.0	83.6	82.5†
MMLU Pro	5-shot	39.8	38.0$^{\diamond}$	39.0$^{\diamond}$	48.3	53.0$^{\diamond}$	54.2†
GPQA	0-shot	32.3	27.0$^{\diamond}$	36.0$^{\diamond}$	36.9	36.0$^{\diamond}$	40.7†
ARC-C	0-shot	85.7	83.4	68.4	93.0	94.8	65.0†
BBH	3-shot	53.4	51.0$^{\diamond}$	60.0$^{\diamond}$	65.5	69	70.8†
HumanEval	pass@1	62.8	72.6	40.2	71.3	80.5	92
GSM8K	5-shot	75.8	75.2/83.7*	68.6	87.0	71.5/94.2*	91.0†
IFEval	0-shot	75.8	80.4	74.3	81.5	87.5	87.8†
BFCL	0-shot	80.7	76.1	-‡	85.5	84.8	85.1†
RealToxicity	avg tox	8.1	-	8.2	6.7	-	-
TruthfulQA	0-shot	54.1	51.5†	50.2	58.3	60.7†	50.4†

Table 2: Jamba-1.5 models obtain similar performance to similarly sized models while enjoying a better throughput and latency. † evaluation run by us. $^{\diamond}$ reported in the HuggingFace OpenLLM leaderboard. ‡ Lacking function calling capabilities. * Strict/flexible evaluation.

[3] In two cases we failed to obtain good results: Mistral-Large-2 fails to obtain good scores on ARC-C despite multiple attempts. LLaMA-3.1 models perform poorly on GSM8K with the standard strict evaluation mode, so we also report for them a flexible evaluation, which allows higher results.

6.2 ChatBot Evaluations

In this section we evaluate the Jamba-1.5 models on two chatbot scenarios: Arena-Hard [22], a set of 500 challenging user queries that uses GPT4-Turbo as a judge, and WildBench [25], which uses GPT4-Turbo as a judge with a length bias mitigation. As Table 3 shows, Jamba-1.5 models obtain excellent reuslts in these evaluations, with Jamba-1.5-Large surpassing LLaMA-3.1 70B, but somewhat trailing behind Mistral-Large-2 123B, which has about 30% more active parameters.

Benchmark	**Jamba-1.5 Mini**	LLaMA-3.1 8B	Gemma-2 9B	**Jamba-1.5 Large**	LLaMA-3.1 70B	Mistral-L-2 123B
Arena-Hard	46.1	21.3	43.2†	65.4	55.7	70.4
Wild-Bench	42.4	33.6†	42.7	48.5	49.8†	56.3†

Table 3: Comparison of Jamba-1.5 models to similarly sized models on chatbot benchmarks. Jamba-1.5 models obtain similar performance with better throughput and latency. † evaluation run by us.

6.3 Long-Context Evaluations

The released model handles context lengths of up to 256K tokens. In this section, we evaluate it on synthetic and naturalistic benchmarks that test its long-context capabilities.

6.3.1 RULER

We evaluate on the RULER benchmark, a set of 13 synthetic tasks aimed to assess long-context capabilities of language models. RULER includes 8 variants of needle-in-a-haystack retrieval tasks [17, 21, 27, 28], including multiple 'needles' [2]. It also has one variable tracking task where a chain of variable bindings should be returned, two aggregation tasks where one needs to return the most common words, and two question-answering tasks, where paragraphs cotraining answers from naturalistic datasets [30, 41] are inserted into random paragraphs to simulate long contexts.

The results are shown in Table 4. Among all publicly available and proprietary models, Jamba-1.5-Mini and Jamba-1.5-Large are the only ones with a confirmed effective length of 256K tokens. Gemini-pro reports good results up to 128K on the original RULER paper. However, we were unable to reproduce these results despite much effort. We examined Gemini-pro generations and noticed the model often fails to answer or generates a refusal. Since the official RULER results are from a preview version, we hypothesize that Gemini-pro had since undergone through updates that have hurt its performacne on RULER.

	Claimed Length	Effective Length	4k	8k	16k	32k	64k	128k	256k	Avg.
Jamba-1.5-Large	256K	**256K**	96.7	96.6	96.4	96.0	95.4	95.1	93.9	95.7
Jamba-1.5-Mini	256K	**256K**	95.7	95.2	94.7	93.8	92.7	89.8	86.1	92.6
Gemini-1.5-pro	1M	>128K	96.7	95.8	96	95.9	95.9	94.4	65.1†	91.4
GPT-4-1106-preview	128K	64K	96.6	96.3	95.2	93.2	87	81.2	-	91.6
LLaMA 3.1 70B	128K	64K	96.5	95.8	95.4	94.8	88.4	66.6	-	89.6
Qwen2 72B	128K	32K	96.9	96.1	94.9	94.1	79.8	53.7	-	85.9
Command-R+	128K	32K	95.6	95.2	94.2	92	84.3	63.1	-	87.4
LLaMA 3.1 8B	128K	32K	95.5	93.8	91.6	87.4	84.7	77	-	88.3
Command-R	128K	32K	93.8	93.3	92.4	89.5	84.9	76	-	88.3
Mistral Large 2	128K	32K	96.2	96.1	95.1	93	78.8	23.7	-	80.5
Mixtral 8x22B	64K	32K	95.6	94.9	93.4	90.9	84.7	31.7	-	81.9
Yi 34B	200K	32K	93.3	92.2	91.3	87.5	83.2	77.3	-	87.5
Phi3 mini 3.8B	128K	32K	92.2	91.5	90.7	87.5	80.6	66.7	-	84.8
Phi3 medium 14B	128K	32K	93.3	93.2	91.1	86.8	78.6	46.1	-	81.5
Mixtral 8x7B	32K	32K	94.9	92.1	92.5	85.9	72.4	44.5	-	80.4
Mistral Nemo 12B	128K	16K	87.8	87.2	87.7	69	46.8	19	-	66.2
DBRX	32K	8K	95.1	93.8	83.6	63.1	2.4	0	-	56.3

Table 4: Comparison of Jamba-1.5 models with other publicly available and proprietary models on the RULER benchmark. Results for other models are from the RULER Github. † Evaluation run by us. Jamba-1.5 models are the only ones with a confirmed effective length of 256K tokens.

6.3.2 Infinite-BENCH

Next we evaluate on ∞BENCH, a dataset designed to evaluate long-context abilities of language models, with an average length of 100K tokens. We focus on two English tasks on understanding long novels: question answering (EN.QA) and multiple-choice question answering (EN.MC). As Table 5 shows, Jamba-1.5 models perform very well in this case, outperforming similarly sized LLaMA-3.1 and Mistral-Large-2 models. (We do not report results with Gemma-2 9B due to its short context window of 8K.)

Benchmark	**Jamba-1.5 Mini**	LLaMA-3.1 8B	Gemma-2 9B	**Jamba-1.5 Large**	LLaMA-3.1 70B	Mistral-L-2 123B
EN.MC	76.9	65.1	-	80.4	78.2	36.9†
EN.QA	40.6	27.1	-	34.9	36.7	-

Table 5: Jamba-1.5 models outperform similarly sized LLaMA-3 and Mistral-Large-2 models in long-context evaluations. † evaluation run by us.

6.4 Multilingual capabilities

We perform a basic evaluation of Jamba-1.5 abilities in non-English langauges. In particular, we report results on the multilingual MMLU dataset [19] as distributed through the LM Evaluation Harness [11]. Table 6 shows the results, where Jamba-1.5-Mini performs similarly or better than its points of comparison. Jamba-1.5-Large is slightly behind its comparable models, but still exhibits good multilingual capabilities.

	Spanish	Portuguese	French	German	Arabic	Italian	Dutch	Avg
Jamba-1.5-Mini	66.3	66.7	65.9	63.8	57.3	65.1	65.0	64.30
LLaMA-3.1-8B	59.5	59.1	59.5	57.2	46.9	58.4	57.2	56.83
Gemma-9B	66.0	59.9	66.7	64.3	55.9	65.8	64.8	63.34
Jamba-1.5-Large	75.5	75.5	75.8	73.9	67.1	75.2	74.6	73.94
LLaMA-3.1-70B	79.5	79.4	79.1	78.4	70.4	79.1	78.4	77.76
Mistral-Large-2	78.7	78.4	78.4	77.4	65.9	78.3	76.2	76.19

Table 6: Comparison of Jamba-1.5 with other models on the multilingual MMLU dataset.

7 Alignment and Safety Considerations

Our approach to alignment of our models is driven by creating transparency between model behavior and customer expectations. Our models default to a business code of conduct based on our participation in industry standards bodies, think tanks and direct experience with our customers. We see this as an ongoing and evolving collaboration. In addition, companies have multiple ways to control model conduct to reflect their individual values and cultures such as additional training and fine tuning, system messages and prompt engineering. Overall, our AI code of conduct is based on the following objectives:

- Align model behavior and output with company values and normative business decorum.
- Clearly state tenets of intended behavior such that errors/bugs are easily discerned.
- Collaborate with Customers and map behavior to their best practices.
- Continuously gather feedback to monitor and actively improve behavior.

In line with our role in an OECD task force to develop a monitoring mechanism for applying the G7 Hiroshima Code of Conduct for Organisations Developing Advanced AI Systems, we have organized our model alignment work with the OECD values-based AI principles:[4] inclusive growth, sustainable development and well-being; human-centered values and fairness; transparency and explainability; robustness, security and safety; and accountability.

[4] https://oecd.ai/en/ai-principles

For each of the first four principles we have detailed behavioral expectations or tenets and examples that can be used to train/align and test for compliance. The principle of accountability is focused on AI21's role in taking responsibility for the behavior of the models. We submit that this accountability is demonstrated primarily through transparency and engagement with customers, regulators and independent 3rd-parties. Our engagement with OECD, Stanford's HELM [23] and FMTI [3] and documents like this demonstrate this commitment, as well as our high ranking on the FMTI (2nd as of May 2024).

In total, we have created 60 tenets that map to the OECD principles. These tenets are stated as directives of behavior for our models to avoid. The full list will be made publicly available.

8 Conclusion

We have presented Jamba-1.5-Large and Jamba-1.5-Mini, two new large-scale models based on the Jamba hybrid Transformer-Mamba architecture. Both models achieve excellent performance in academic benchmarks, chatbot evaluations, and long-context evaluations, while offering improved latency and throughput, especially for long contexts. We release the model weights for use by the community in hopes that others build on this technology.

Contributions

Pre- and Post-Training
Alan Arazi
Barak Lenz*
Chen Almagor
Dan Padnos*
Daniel Gissin*
Daniel Jannai
Dor Muhlgay
Edden M Gerber
Erez Safahi
Gal Cohen
Gal Shachaf
Hofit Bata
Inbal Magar
Itay Dalmedigos
Jhonathan Osin*
Matan Danos
Michael Gokhman
Nir Ratner
Noam Gat
Noam Rozen
Omer Antverg
Omri Abend
Opher Lieber*
Orit Cohavi
Raz Alon
Shaked Meirom
Tom Braude
Uriya Pumerantz
Yonatan Belinkov
Yuval Globerson
Yuval Peleg Levy

Serving & Infrastructure
Amir Bergman
Avshalom Manevich
Barak Peleg
Elad Dolev
Eran Krakovsky
Erez Schwartz
Haim Rozenblum
Mor Zusman
Oded Fried
Roman Glozman
Shahar Lev
Tomer Asida
Yehoshua Cohen

Data
Ben Aviram
Dor Zimberg
Ido Blass
Ohad Leshno
Rom Gilad
Tom Ben Gal

Evaluation
Clara Fridman
Julie Fadlon
Maria Rozman
Naama Gidron
Ro'i Belson
Tal Ness

Project & Product Management
Or Dagan*
Roi Cohen
Shaked Meirom*
Tal Delbari
Yoav Shoham

*Project leads

References

[1] Glaive AI. Glaive function calling v2. https://huggingface.co/datasets/glaiveai/glaive-function-calling-v2.

[2] Simran Arora, Sabri Eyuboglu, Aman Timalsina, Isys Johnson, Michael Poli, James Zou, Atri Rudra, and Christopher Re. Zoology: Measuring and improving recall in efficient language models. In *The Twelfth International Conference on Learning Representations*.

[3] Rishi Bommasani, Kevin Klyman, Sayash Kapoor, Shayne Longpre, Betty Xiong, Nestor Maslej, and Percy Liang. The foundation model transparency index v1.1: May 2024. 2024.

[4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[5] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

[6] Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. *ArXiv*, abs/2405.21060, 2024.

[7] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[8] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[9] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. OPTQ: Accurate quantization for generative pre-trained transformers. In *The Eleventh International Conference on Learning Representations*, 2023.

[10] Trevor Gale, Deepak Narayanan, Cliff Young, and Matei Zaharia. MegaBlocks: Efficient Sparse Training with Mixture-of-Experts. *Proceedings of Machine Learning and Systems*, 5, 2023.

[11] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation, 07 2024.

[12] Samuel Gehman, Suchin Gururangan, Maarten Sap, Yejin Choi, and Noah A Smith. Realtoxicityprompts: Evaluating neural toxic degeneration in language models. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 3356–3369, 2020.

[13] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

[14] Albert Gu, Karan Goel, and Christopher Re. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*, 2021.

[15] Albert Gu, Isys Johnson, Karan Goel, Khaled Saab, Tri Dao, Atri Rudra, and Christopher Ré. Combining recurrent, convolutional, and continuous-time models with linear state space layers. *Advances in neural information processing systems*, 34:572–585, 2021.

[16] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2020.

[17] Greg Kamradt. Needle in a haystack - pressure testing llms. https://github.com/gkamradt/LLMTest_NeedleInAHaystack/, 2023.

[18] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

[19] Viet Lai, Chien Nguyen, Nghia Ngo, Thuat Nguyen, Franck Dernoncourt, Ryan Rossi, and Thien Nguyen. Okapi: Instruction-tuned large language models in multiple languages with reinforcement learning from human feedback. In Yansong Feng and Els Lefever, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 318–327, Singapore, December 2023. Association for Computational Linguistics.

[20] Barak Lenz, Raz Along, Noam Rozen, Omri Abend, Yonatan Belinkov, Kevin Leyton-Brown, and Yoav Shoham. Structured data as a key element of ai systems: A test case on table understanding. In *Compound AI Systems Workshop*, 2025.

[21] Dacheng Li, Rulin Shao, Anze Xie, Ying Sheng, Lianmin Zheng, Joseph Gonzalez, Ion Stoica, Xuezhe Ma, and Hao Zhang. How long can context length of open-source llms truly promise? In *NeurIPS 2023 Workshop on Instruction Tuning and Instruction Following*, 2023.

[22] Tianle Li*, Wei-Lin Chiang*, Evan Frick, Lisa Dunlap, Banghua Zhu, Joseph E. Gonzalez, and Ion Stoica. From live data to high-quality benchmarks: The arena-hard pipeline, April 2024.

[23] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, Benjamin Newman, Binhang Yuan, Bobby Yan, Ce Zhang, Christian Cosgrove, Christopher D. Manning, Christopher R'e, Diana Acosta-Navas, Drew A. Hudson, E. Zelikman, Esin Durmus, Faisal Ladhak, Frieda Rong, Hongyu Ren, Huaxiu Yao, Jue Wang, Keshav Santhanam, Laurel J. Orr, Lucia Zheng, Mert Yuksekgonul, Mirac Suzgun, Nathan S. Kim, Neel Guha, Niladri S. Chatterji, O. Khattab, Peter Henderson, Qian Huang, Ryan Chi, Sang Michael Xie, Shibani Santurkar, Surya Ganguli, Tatsunori Hashimoto, Thomas F. Icard, Tianyi Zhang, Vishrav Chaudhary, William Wang, Xuechen Li, Yifan Mai, Yuhui Zhang, and Yuta Koreeda. Holistic evaluation of language models. *Annals of the New York Academy of Sciences*, 1525:140 – 146, 2023.

[24] Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, Erez Safahi, Shaked Haim Meirom, Yonatan Belinkov, Shai Shalev-Shwartz, Omri Abend, Raz Alon, Tomer Asida, Amir Bergman, Roman Glozman, Michael Gokhman, Avshalom Manevich, Nir Ratner, Noam Rozen, Erez Shwartz, Mor Zusman, and Yoav Shoham. Jamba: A hybrid transformer-mamba language model. *ArXiv*, abs/2403.19887, 2024.

[25] Bill Yuchen Lin, Yuntian Deng, Khyathi Chandu, Faeze Brahman, Abhilasha Ravichander, Valentina Pyatkin, Nouha Dziri, Ronan Joseph Le Bras, and Yejin Choi. Wildbench: Benchmarking llms with challenging tasks from real users in the wild. *ArXiv*, abs/2406.04770, 2024.

[26] Stephanie Lin, Jacob Hilton, and Owain Evans. Truthfulqa: Measuring how models mimic human falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3214–3252, 2022.

[27] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2023.

[28] Amirkeivan Mohtashami and Martin Jaggi. Random-access infinite context length for transformers. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.

[29] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36, 2024.

[30] Pranav Rajpurkar, Robin Jia, and Percy Liang. Know what you don't know: Unanswerable questions for squad. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 784–789, 2018.

[31] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. GPQA: A graduate-level Google-proof Q&A benchmark. *arXiv preprint arXiv:2311.12022*, 2023.

[32] Victor Sanh, Albert Webson, Colin Raffel, Stephen Bach, Lintang Sutawika, Zaid Alyafeai, Antoine Chaffin, Arnaud Stiegler, Arun Raja, Manan Dey, et al. Multitask prompted training enables zero-shot task generalization. In *International Conference on Learning Representations*.

[33] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[34] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.

[35] Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc Le, Ed Chi, Denny Zhou, et al. Challenging BIG-Bench tasks and whether chain-of-thought can solve them. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 13003–13051, 2023.

[36] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[37] Roger Waleffe, Wonmin Byeon, Duncan Riach, Brandon Norick, Vijay Anand Korthikanti, Tri Dao, Albert Gu, Ali Hatamizadeh, Sudhakar Singh, Deepak Narayanan, Garvit Kulshreshtha, Vartika Singh, Jared Casper, Jan Kautz, Mohammad Shoeybi, and Bryan Catanzaro. An empirical study of mamba-based language models. *ArXiv*, abs/2406.07887, 2024.

[38] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, et al. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. *arXiv preprint arXiv:2406.01574*, 2024.

[39] Jason Wei, Maarten Bosma, Vincent Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M Dai, and Quoc V Le. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations*.

[40] Fanjia Yan, Huanzhi Mao, Charlie Cheng-Jie Ji, Tianjun Zhang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Berkeley function calling leaderboard. 2024.

[41] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Conference on Empirical Methods in Natural Language Processing*, 2018.

[42] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023.